

Szakdolgozói leadandó csomag

AI-val fejlesztett termékminőségű alkalmazások
Dokumentációs követelmények és pontozási rendszer

Verzió: 1.2

Dátum: 2026-02-23

Készült a szakdolgozói (termékminőség + AI-first) fejlesztések egységes értékeléséhez.

Tartalomjegyzék

Tartalomjegyzék	2
1. Áttekintés.....	7
1.1 Keretek és alapelvek	7
1.2 Fogalmak.....	7
1.3 Módszertan: minimális papír, maximális bizonyíték.....	7
2. Kötelező repo-struktúra és általános elvárások.....	8
2.1 Egy repo - minden benne	8
2.2 Futtathatóság (reprodukálhatóság) - kötelező	8
2.3 Secrets és konfiguráció - zéró tolerancia	8
2.4 Evidence szabály: minden fontos állítás linkelhető legyen	9
2.5 Dokumentáció minőségi minimum.....	9
3. Kötelező dokumentumok - részletes követelmények.....	9
3.A README.md - belépési pont és futtathatóság	9
Cél	9
Kötelező tartalom (minimál struktúra)	9
Elvárt bizonyítékok.....	10
Minimum vs. kiemelkedő szint	10
Tipikus hibák (amitől pontot veszítesz).....	10
Ajánlott AI-munkafolyamat.....	10
3.B Product Vision + Scope Contract - mit építünk és mit nem	10
Cél	10
Vision dokumentum (docs/01_product/vision.md).....	10
Scope Contract (docs/01_product/scope_contract.md)	10
Elvárt bizonyítékok.....	11
Minimum vs. kiemelkedő szint	11
Tipikus hibák	11
Ajánlott AI-munkafolyamat.....	11
3.C Capability Map - a termék képességeinek térképe.....	11
Cél	11
Kötelező minimum.....	11
Ajánlott capability kategóriák (ötletek)	12

Capability Map táblázat - kötelező oszlopok	12
Elvárt bizonyítékok.....	12
Minimum vs. kiemelkedő szint	12
Tipikus hibák	12
Ajánlott AI-munkafolyamat.....	12
3.D UX Flows - használhatóság és felhasználói útvonalak	13
Cél	13
Kötelező tartalom	13
Elvárt bizonyítékok.....	13
Minimum vs. kiemelkedő szint	13
Tipikus hibák	13
Ajánlott AI-munkafolyamat.....	13
3.E Architektúra (C4) + ADR - mérnöki döntések és tradeoffok	13
Cél	13
Kötelező C4 szintek	14
Diagram követelmények	14
Quality attributes (docs/02_architecture/quality_attributes.md)	14
ADR követelmények (docs/02_architecture/adr/)	14
ADR sablon - kötelező mezők	14
Elvárt bizonyítékok.....	15
Minimum vs. kiemelkedő szint	15
Tipikus hibák	15
Ajánlott AI-munkafolyamat.....	15
3.F API + Data model + Error handling - szerződés és stabilitás	15
Cél	15
API dokumentáció (docs/03_design/api.md)	15
Data model (docs/03_design/data_model.md).....	16
Error handling (docs/03_design/error_handling.md).....	16
Elvárt bizonyítékok.....	16
Minimum vs. kiemelkedő szint	16
Tipikus hibák	16
Ajánlott AI-munkafolyamat.....	16

3.G Teszt stratégia + Teszt riport - bizonyítsd, hogy igaz	17
Cél	17
Teszt stratégia (docs/04_quality/test_strategy.md)	17
Minimum teszt mennyiség és összetétel	17
Teszt riport (docs/04_quality/test_report.md)	17
Elvárt bizonyítékok.....	17
Minimum vs. kiemelkedő szint	17
Tipikus hibák	18
Ajánlott AI-munkafolyamat.....	18
3.H Security + Privacy + Licensing - felelős termék.....	18
Cél	18
Threat model (docs/05_security_ops/threat_model.md)	18
Privacy + adatkezelés (docs/05_security_ops/privacy_licensing.md)	18
Kötelező security baseline a kódban.....	19
Elvárt bizonyítékok.....	19
Minimum vs. kiemelkedő szint	19
Tipikus hibák	19
Ajánlott AI-munkafolyamat.....	19
3.I Deploy + Runbook + Observability - üzemeltethető rendszer	19
Cél	19
Deploy runbook (docs/05_security_ops/deploy_runbook.md)	19
Runbook (operáció) - minimum incident forgatókönyvek.....	20
Observability (docs/05_security_ops/observability.md)	20
Elvárt bizonyítékok.....	20
Minimum vs. kiemelkedő szint	20
Tipikus hibák	20
Ajánlott AI-munkafolyamat.....	20
3.J AI dokumentáció - átlátható és verifikált AI-first fejlesztés.....	21
Cél	21
Kötelező AI artefaktok	21
AI Manifest - tartalom és elvárás.....	21
Prompt Log - tartalom és forma	21

Verification Log - a legfontosabb rész.....	21
AI használat minőségi szabályai (kötelező betartani)	22
Elvárt bizonyítékok.....	22
Minimum vs. kiemelkedő szint	22
4. Értékelés és pontozás	22
4.1 Kapu feltételek (ha nincs meg, plafon vagy levonás).....	22
4.2 Pontozási rubrika (100 pont)	23
4.3 Részletes értékelési leírás kategóriánként.....	24
A) Product & Scope (12 pont)	24
B) Capability Breadth (10 pont)	24
C) Architecture & Decisions (13 pont)	25
D) Engineering Quality (15 pont)	25
E) Testing & Quality Gates (15 pont)	26
F) DevOps & Operations (15 pont).....	26
G) Security, Privacy, Licensing (10 pont).....	27
H) AI Engineering Maturity (10 pont)	27
4.4 Bónusz (0-10 pont, opcionális).....	28
4.5 Önértékelés (kötelező melléklet a leadásban).....	28
5. Végállapot: leadási elvárások.....	29
5.1 Leadási csomag definíciója	29
5.2 Release-ready elfogadási kritériumok	29
5.3 Technikai mélység elvárások területenként	30
5.3.1 Futtathatóság, build és konfiguráció	30
5.3.2 Funkcionalitás és UX	30
5.3.3 Architektúra, modulhatárok és bővíthetőség.....	31
5.3.4 Adatmodell és API szerződés	31
5.3.5 Tesztelés és regresszióvédelem	32
5.3.6 CI/CD, kiadás és üzemeltethetőség.....	32
5.3.7 Megfigyelhetőség és diagnosztika	33
5.3.8 Biztonság, adatvédelem és licenc	33
5.3.9 AI engineering és verifikáció.....	34
6. Önellenzés: készütség és termékminőség mérése	34

6.1 Gyors ellenőrzés 15 perc alatt	35
6.2 Kapu checklist	36
6.3 100 pontos készülségi scorecard	36
A) Product és scope - 12 pont	36
B) Képesség-szélesség - 10 pont	37
C) Architektúra és döntések - 13 pont	37
D) Engineering minőség - 15 pont	38
E) Tesztelés és minőségi kapuk - 15 pont	38
F) DevOps és üzemeltetés - 15 pont	39
G) Security, privacy, licenc - 10 pont.....	39
H) AI engineering érettség - 10 pont	40
Bónusz - 10 pont	40
6.4 Pontszám értelmezése	41
7. Mellékletek: sablonok.....	41
7.1 ADR sablon (docs/02_architecture/adr/000X-title.md)	41
7.2 Threat model sablon (STRIDE táblázat).....	42
7.3 Verification log sablon	42
7.4 Teszt riport sablon (rövid).....	42
7.5 Demo script sablon	43
7.6 Metrics sablon	43

1. Áttekintés

Ez a dokumentum egységesen leírja, hogy a szakdolgozók (vagy szakdolgozati jellegű fejlesztést készítő) milyen dokumentációt adjanak le, ha AI-val fejlesztenek termékminőségű alkalmazást. A dokumentáció célja nem az, hogy sok oldal legyen, hanem hogy a termék minősége, működése és üzemeltethetősége bizonyítható és újraépíthető legyen (evidence-first szemlélet).

1.1 Keretek és alapelvek

Elvart ráfordítás: 600 óra munka. A dokumentum nem ír elő kötelező ütemezést vagy sprintet: a leadás VÉGÁLLAPOTÁT (mit kell a végére szállítani) és a folyamatos önellenőrzés módját rögzíti.

A hallgatók minden artefaktot AI segítségével készítenek, de a felelősség a végső minőségért és helyességért a hallgatóé.

A dokumentáció és a kód együtt alkot egy leadandó csomagot. A leadandó csomag mindig a repo-ban él (Docs-as-Code), verziózott, és futtatható/reprodukálható.

Termékminőség = A termékminőség ebben a kontextusban azt jelenti, hogy az alkalmazás:

- felhasználói értéket ad (világos cél, MVP, elfogadási kritériumok).
- megbízhatóan működik (tesztek, hibakezelés, regresszióvédelem).
- üzemeltethető (deploy leírás, konfiguráció, logok, monitorozhatóság, runbook).
- biztonságos és felelős (secrets kezelése, threat model, adatvédelem, licenc).
- karbantartható és bővíthető (architektúra, döntések, moduláris felépítés).
- AI-használata átlátható és verifikált (promptok és validációk dokumentálva).

1.2 Fogalmak

A dokumentumban használt fogalmak röviden:

- Leadandó csomag: egyetlen Git repo, benne a kód, a dokumentáció és a futtathatósághoz szükséges fájlok.
- Docs-as-Code: a dokumentáció Markdown-ban (vagy generált site-ban) a repo része; review-zható, verziózott.
- Evidence (bizonyíték): olyan hivatkozás vagy artefakt, ami alátámasztja az állítást (tesztlog, CI run, screenshot, mérés).
- ADR (Architecture Decision Record): rövid, verziózott döntésnapló; miért ezt választottad, milyen tradeoffokkal.
- Quality attribute: nemfunkcionális minőségi elvárás (pl. teljesítmény, rendelkezésre állás, biztonság, maintainability).
- AI verifikáció: az AI által javasolt állítások/megoldások ellenőrzése (teszttel, méréssel, forrással, kísérellettel).

1.3 Módszertan: minimális papír, maximális bizonyíték

A dokumentáció minden pontján az a kérdés: 'Ha ezt egy külsős mérnök elolvassa, képes-e (1) megérteni, (2) futtatni, (3) értékelni, hogy termékminőség-e, és (4) továbbfejleszteni?'

Ahol állítást teszel (pl. 'biztonságos', 'gyors', 'offline működik'), ott kötelező mellé tenni a bizonyítékot (teszt, mérés, screenshot, log, CI riport).

2. Kötelező repo-struktúra és általános elvárások

2.1 Egy repo - minden benne

A leadás egyetlen Git repo linkje. A repo-n belül legyen minden, ami a megértéshez és futtatáshoz kell (kivéve a titkok/secrets). A repo-n kívüli drive linkek, külön dokumentumok csak kiegészítők lehetnek, de a hivatalos leadandó a repo tartalma.

Struktúra: Ajánlott (de nem kötelező) struktúra:

```
/
  README.md
  docs/
    00_index.md
    01_product/...
    02_architecture/...
    03_design/...
    04_quality/...
    05_security_ops/...
    06_release/...
    07_ai/...
  (alkalmazás kódja)
  (tesztek)
  (CI/CD pipeline fájlok)
```

2.2 Futtathatóság (reprodukálhatóság) - kötelező

A futtathatóság azt jelenti, hogy egy oktató/peer egy szűz gépen, a README alapján, reális erőfeszítéssel (max. 15 perc) el tud jutni oda, hogy a rendszer elindul, és legalább egy alap funkció kipróbálható.

Kötelező: Kötelező elemek:

- Konkrét parancsok: build, run, test (ne csak 'indítsd el').
- Környezeti változók listája és jelentése (ENV template: .env.example vagy hasonló).
- Adatbázis/függőségek indítása (docker compose ajánlott).
- Minimális seed/adat inicializálás (ha kell).
- Hibakeresés: gyakori hibák és megoldások (FAQ jelleggel).

2.3 Secrets és konfiguráció - zéró tolerancia

Titkok (token, jelszó, API key) nem kerülhetnek a repo-ba. Ez termékminőség alapkövetelmény.

Példák: Elvárt megoldások példák:

- .env.example + dokumentáció, hogy milyen értékeket vár a rendszer.
- CI-ban secrets store használata (GitHub Actions secrets, GitLab CI variables, stb.).
- Konfiguráció rétegek: default config + environment override.

Értékelés: ha a repo-ban secret található, az leadás elfogadhatatlan javításig (lásd pontozás).

2.4 Evidence szabály: minden fontos állítás linkelhető legyen

Evidence: A dokumentációban minden lényegi állításhoz legyen evidence:

- CI futás linkje / badge / képernyőkép (tesztek, lint, build).
- Teszt riport (konkrét futtatás eredménye).
- Képernyőképek vagy rövid GIF a fő user flow-khoz (ha UI van).
- Mérés (pl. egyszerű load test eredmény) ha teljesítményt állítasz.
- Log minták és health endpoint válaszok (operálhatóság).

2.5 Dokumentáció minőségi minimum

A dokumentációt úgy kell megírni, mintha egy csapatban adnod kellene át a projektet.

Minimum: Minimum elvárások:

- Konzisztens szerkezet (címek, alfejezetek, listák).
- Nincsenek 'TODO' vagy üres fejezetek a leadásban (ha hiányzik valami, azt 'Ismert hiányosságok' alatt írd le).
- A linkek működnek (repo-n belüli relatív linkek preferáltak).
- Minden diagram olvasható (ne legyen elmosódott screenshot).
- A dokumentáció nem ellentmondásos a kóddal (ha változik a kód, frissül a doku is).

3. Kötelező dokumentumok - részletes követelmények

Ebben a fejezetben minden kötelező dokumentumhoz megadjuk: (1) cél, (2) kötelező tartalom, (3) elvárt bizonyítékok, (4) minimum vs. kiemelkedő szint, (5) tipikus hibák, (6) ajánlott AI-munkafolyamat (hogyan készítsd AI-val, úgy hogy igaz is legyen).

3.A README.md - belépési pont és futtathatóság

Cél

A README az a fájl, amit az első 30 másodpercben elolvasunk. Ez dönti el, hogy a projekt komolyan vehető-e, és 15 percen belül fut-e. Termékminőségben a README a 'csomagolás' része.

Kötelező tartalom (minimál struktúra)

1. Projekt neve + 1 mondatos értékígéret (kinek, milyen problémára).
2. Fő képességek / use case-ek (3-6 bullet).
3. Tech stack röviden (frontend, backend, DB, infra).
4. Quickstart (lokál): parancsok sorrendben, expected outputtal (mit kell látnia a végén).
5. Teszt futtatás: 1 parancs vagy rövid listával (unit / integration / e2e).
6. Konfiguráció: szükséges env változók + .env.example, és hogyan töltsd ki.
7. Demo / kipróbálás: URL vagy lokál portok, teszt user (ha releváns).
8. Dokumentáció belső link (docs index).

9. Licenc / harmadik fél komponensek röviden (ha van).

Elvárt bizonyítékok

- README-ben szerepeljen egy CI badge vagy legalább egy friss teszt futás képernyőkép/link.
- Quickstart lépések után készült screenshot (pl. futó UI vagy health endpoint válasz).

Minimum vs. kiemelkedő szint

- Minimum: működő, pontos quickstart, teljes parancslista, nincs hiányzó lépés.
- Kiemelkedő: troubleshooting (gyakori hibák), platform specifikus különbségek (Windows/Mac/Linux), és egy '1 perces demo' rész (mit próbáljak ki először).

Tipikus hibák (amitől pontot veszítesz)

- Nem fut a quickstart (hiányzó parancs, rossz port, hiányzó seed).
- A README állítása ellentmond a kódnak (pl. 'Dockerrel fut', de nincs compose).
- Beégetett URL-ek / tokenek / jelszavak a README-ben.
- Túl hosszú, de a lényeget nem mondja (a quickstart legyen az elején).

Ajánlott AI-munkafolyamat

10. Készíts egy nyers README vázlatot AI-val (csak címek + bullet).
11. Futtasd végig a quickstartot egy tiszta környezetben (akár konténerben), és javítsd a README-t a valós lépésekhez.
12. Add a README-hez a leggyakoribb 3 hibát (AI segítségével gyűjts ötleteket), majd VALIDÁLD (tényleg előjön?).
13. Tedd be a CI badge-et és a teszt parancsokat, ellenőrizd, hogy kattinthatók és működnek.

3.B Product Vision + Scope Contract - mit építünk és mit nem

Cél

A Vision és a Scope Contract biztosítja, hogy a projekt ne legyen 'random feature gyűjtemény'. Ez ad termék szemléletet: világos cél, világos felhasználó, és mérhető definíciója annak, hogy mi kész el.

Vision dokumentum (docs/01_product/vision.md)

Elvárás: 1-2 oldal. Rövid, de tömör és döntéskényszerítő.

Fejezetek: Kötelező fejezetek:

14. Probléma leírás (1 bekezdés): mi fáj jelenleg? miért most?
15. Célfelhasználó (persona): 1-2 persona, célok és frusztrációk.
16. Értékajánlat (value proposition): miben jobb a megoldásod? (konkrét, nem marketing).
17. Siker definíció: 1 North Star metric + 2-3 guardrail metric (lásd metrics.md).
18. Non-goals: mit nem fogsz megoldani ebben a scope-ban (legalább 3 tétel).
19. Kockázatok és bizonytalanságok: 3-5 kockázat, mitigációval.

Scope Contract (docs/01_product/scope_contract.md)

Elvárás: 1-2 oldal. Ez a projekt 'szerződése' az értékeléshez.

Kötelező: Kötelező elemek:

20. MVP user story lista (3-6): mindegyikhez rövid leírás.
21. Elfogadási kritériumok story-nként (Given-When-Then vagy bullet).
22. Stretch (opcionális) lista (2-4): ha idő van, ezek jöhetnek.
23. Korlátok: idő, adat, platform, külső API-k (ha vannak).
24. Kész definíciója (Definition of Done) a release-re: mi kell a leadáshoz.

Elvárt bizonyítékok

- Az MVP story-k mindegyikére legyen legalább 1 automata teszt vagy demonstrálható flow.
- A 'Siker definíció' metrikákhoz legyen mérési terv (nem kell éles analitika, de legyen 'hogyan mérném').

Minimum vs. kiemelkedő szint

- Minimum: tiszta MVP lista + elfogadási kritériumok + non-goals.
- Kiemelkedő: kockázatkezelés (pl. külső API kiesik), és mérhető, realisztikus metrika definíciók.

Tipikus hibák

- Vision tele van általánosságokkal ('felhasználóbarát'), de nincs konkrét persona/probléma.
- Scope túl nagy (10+ user story), nincs non-goals, nincs prioritás.
- Elfogadási kritériumok hiányoznak, ezért nem objektív az értékelés.

Ajánlott AI-munkafolyamat

25. Add meg AI-nak: célterület, céluser, platform, és kérd, hogy generáljon 3 persona-t. Válassz 1-2-t, majd kézzel finomítsd.
26. Kérd AI-tól MVP user story javaslatokat, de TE vágd le 3-6-ra. Ez a scope fegyelem része.
27. Kérd AI-tól acceptance criteria mintákat (Given-When-Then), majd validáld: tényleg tesztelhetőek?
28. Írd le non-goals-t tudatosan: minden, amit 'jó lenne' de most nem fér bele.

3.C Capability Map - a termék képességeinek térképe

Cél

A capability map megmutatja, hogy a termék milyen képességeket ad, és ebből mennyi a 'felhasználói érték' és mennyi a 'termékesítés' (productization: minőség, üzemeltetés, biztonság). Ez védi a projektet attól, hogy csak egyetlen látványos feature legyen, de hiányozzanak a termékminőségi alapok.

Kötelező minimum

- Legalább 6 capability (képesség).
- Ebből legalább 3 'Value' (felhasználói érték) és legalább 3 'Productization' (termékminőség) kategóriában.
- Minden 'Done' capability-hez: evidence link + teszt link.
- A capability-k neve legyen cselekvő, világos (pl. 'Bejelentkezés és jogosultságok', nem 'Auth').

Ajánlott capability kategóriák (ötletek)

- Value: keresés/szűrés, tartalom létrehozás, megosztás, értesítés, riport, workflow automatizáció.
- Productization: auth/RBAC, offline/retry/queue, CI/CD, logging/metrics/healthcheck, backup/migráció, rate limit, accessibility, i18n.

Capability Map táblázat - kötelező oszlopok

A táblázatot a docs/01_product/capability_map.md fájlban készítsd el.

Capability	Kategória (Value/Productization)	Evidence (link)	Teszt (link)	Státusz (Done/Partial/Planned)
Felhasználói regisztráció és login	Productization	CI run + screenshot	auth.integration.test	Done
Tétel létrehozása és szerkesztése	Value	UI screenshot/GIF	e2e_create_edit	Done
Audit log és esemény naplózás	Productization	log minták	unit_audit_log	Partial

Elvárt bizonyítékok

- Evidence link: repo-n belüli relatív link (pl. docs/..), vagy CI run link, vagy screenshot a docs alatt.
- Teszt link: konkrét teszt fájl/teszt suite hivatkozás (ne csak annyi: 'van teszt').

Minimum vs. kiemelkedő szint

- Minimum: 6 capability, kitöltött evidence + teszt minden Done-hoz.
- Kiemelkedő: capability-khez mérhető metrikák (pl. 'login success rate'), és kockázatok (pl. 'offline sync conflict').

Tipikus hibák

- A map csak feature-öket tartalmaz, productization nincs benne.
- Done státuszhoz nincs bizonyíték vagy nincs teszt.
- Capability-k túl technikaiak (pl. 'Redis cache') ahelyett, hogy képességet neveznének meg.

Ajánlott AI-munkafolyamat

29. Kérd AI-tól: listázzon 10 value capability-t és 10 productization capability-t a domain alapján.
30. Válaszd ki a 6-8 legfontosabbat, majd fogalmazd át képesség nyelvre (user value).
31. Minden capability-hez kérj AI-tól 'evidence ötleteket' (mit tudsz linkelni), majd valósítsd meg és linkeld.

3.D UX Flows - használhatóság és felhasználói útvonalak

Cél

A termékminőség egyik legfontosabb része, hogy a felhasználó végig tud menni a fő feladatain. A UX flow dokumentum nem UI dizájn pályamunka, hanem mérnöki leírás arról, hogy a felhasználó milyen lépésekben éri el a célját, és mi történik hibák/üres állapotok esetén.

Kötelező tartalom

32. 2-3 fő user flow (pl. onboarding -> core action -> visszaigazolás).
33. Minden flow-hoz: cél, előfeltételek, lépések (1-2 mondat/lépés), siker kritérium.
34. Hibakezelés UX: legalább 3 tipikus hibahelyzet leírva (pl. hálózat nincs, rossz jelszó, üres találat).
35. Empty state: mit lát a user, ha nincs adat.
36. Alap accessibility: szövegek olvashatósága, kontraszt, billentyűzet/screen reader minimum (platformtól függően).

Elvárt bizonyítékok

- Képernyőképek vagy rövid GIF-ek minden fő flow-hoz (a docs alatt tárolva).
- Legalább 1 e2e teszt vagy UI teszt a legfontosabb flow-ra (ha UI van).
- Hibakezelés bizonyíték: screenshot, ahol hibaüzenet és recovery látszik.

Minimum vs. kiemelkedő szint

- Minimum: 2 fő flow + error/empty state dokumentálva, screenshotokkal.
- Kiemelkedő: edge case-ek (pl. duplikált mentés, konfliktus), és világos mikrocopy (nem technikai hibaüzenet).

Tipikus hibák

- Csak a happy path van leírva, hibák és üres állapotok nélkül.
- A flow leírás nincs összhangban a valós UI-val (screenshot más állapotot mutat).
- A hibaüzenetek fejlesztői nyelven vannak (stack trace, HTTP 500).

Ajánlott AI-munkafolyamat

37. Kérd AI-tól: generáljon user flow-kat a persona és MVP story-k alapján.
38. Válaszd ki a 2-3 legfontosabbat, majd implementálás után készíts screenshotokat a valós UI-ról.
39. Kérd AI-tól: listázzon 10 tipikus hiba/empty state helyzetet a domainben, és válassz belőle legalább 3-at, amit tényleg lekezelysz.
40. Validáld a hibaüzeneteket: külső szemmel olvasható? Mit tegyen a user?

3.E Architektúra (C4) + ADR - mérnöki döntések és tradeoffok

Cél

Az architektúra dokumentáció azt mutatja meg, hogy a rendszered hogyan épül fel, és miért így. AI-val írt kódnál különösen fontos, hogy a rendszer ne legyen 'véletlenszerűen összehegesztett', hanem tudatos

döntések mentén felépített. Az ADR-ek azért kellenek, hogy a jövőben érthető legyen: miért ezt választottad, és milyen következményei vannak.

Kötelező C4 szintek

Minimum elvárás: Context + Container diagram. Ha komplexebb a rendszer, akkor Component diagram is.

- 41. C4-Context: a rendszered környezete - felhasználók, külső rendszerek, integrációk.
- 42. C4-Container: futtatható egységek - frontend, backend, DB, queue, third-party szolgáltatás.
- 43. C4-Component (opcionális, de ajánlott): backend belső komponensek/modulok, felelősségek.

Diagram követelmények

- Olvasható (ne legyen pixeles screenshot).
- Minden doboznak legyen neve és felelőssége (1 rövid mondat).
- Minden kapcsolatnak legyen iránya és típusa (HTTP, gRPC, async event, DB connection).
- Jelöld, hol van auth boundary, hol tárolsz adatot, és hol vannak külső függőségek.

Quality attributes (docs/02_architecture/quality_attributes.md)

A quality attributes fejezetben 5-8 nemfunkcionális elvárást kell megfogalmazni, és ezekből legalább 2-t konkrét 'quality attribute scenario' formában.

Sablon: Quality attribute scenario sablon (kötelező elemek):

- Forrás (source): ki/mi indítja az eseményt (felhasználó, rendszer, támadó).
- Stimulus: mi történik (kérés, hiba, terhelés, kiesés).
- Környezet: milyen állapotban (normál, csúcsidő, offline, degradált).
- Artefakt: melyik komponens érintett.
- Válasz: hogyan reagál a rendszer (fallback, retry, rate limit, error message).
- Mérőszám: mi a mérhető elvárás (pl. p95 latency < 300ms).

ADR követelmények (docs/02_architecture/adr/)

Minimum 5 ADR. Minden ADR egy konkrét döntésről szól. Példák jó ADR témákra:

- Auth megoldás (JWT, session, OAuth) és következmények.
- Állapotkezelés / architektúra minta (Clean Architecture, MVVM, hexagonal).
- Adatbázis választás és migráció stratégia.
- API design (REST vs GraphQL), verziózás, hibamodell.
- CI pipeline döntések (quality gates, test types).
- Logolás/observability stratégia (structured logging, trace id).

ADR sablon - kötelező mezők

- 44. Cím + dátum + státusz (Proposed/Accepted/Deprecated).
- 45. Context: mi a probléma, milyen kényszerek vannak.
- 46. Decision: mit választottál (egy mondatban).
- 47. Alternatívák: legalább 2 alternatíva röviden.

48. Következmények: pozitív és negatív hatások, kockázatok.

49. Verification: hogyan ellenőrizted vagy ellenőrzöd, hogy jó döntés volt (teszt/mérés/PoC).

Elvárt bizonyítékok

- Diagramok linkelve és a kódban visszaköthető modulnévvel (pl. 'api-service', 'mobile-app').
- ADR-ben a 'Verification' részhez evidence (PoC branch, benchmark, teszt).
- Deployment view: legalább 1 ábra/diagram, hol fut mi (lokál, docker, cloud).

Minimum vs. kiemelkedő szint

- Minimum: Context+Container diagram + 5 ADR + 5-8 quality attribute lista.
- Kiemelkedő: Component diagram, és legalább 2 quality attribute scenario mérőszámmal; ADR-ekben tradeoffok őszintén leírva.

Tipikus hibák

- Diagram túl részletes (minden osztály rajta), vagy túl homályos (csak 2 doboz).
- ADR csak 'mi' van, de a 'miért' és az alternatívák hiányoznak.
- Quality attributes üres buzzword lista, mérőszámok és scenario nélkül.

Ajánlott AI-munkafolyamat

50. Kérd AI-tól C4 diagram tartalmát (dobozok + kapcsolatok), majd te rajzold meg (Mermaid, PlantUML, draw.io).
51. Minden nagy döntésnél kérdezd AI-t: 'Milyen alternatívák vannak és mik a tradeoffok?', majd írd ADR-be.
52. A 'Verification' részhez kérj AI-tól ellenőrzési ötleteket (benchmark, teszt), de te futtasd és dokumentáld az eredményt.

3.F API + Data model + Error handling - szerződés és stabilitás

Cél

A termékminőség egyik alapja a stabil szerződés: hogyan kommunikálnak a komponensek, milyen adatot tárolunk, és hogyan kezeljük a hibákat konzisztensen. Ez különösen fontos, ha a fejlesztést AI segíti, mert könnyen keletkezik inkonzisztens API és adatformátum.

API dokumentáció (docs/03_design/api.md)

Ha van backend/API, akkor kötelező egy API dokumentum. Preferált forma: OpenAPI (Swagger) export vagy link.

Kötelező: Kötelező tartalom:

53. Base URL / lokál URL + verziózás (pl. /api/v1).
54. Auth mód (token/session), és jogosultságok (melyik endpoint kié).
55. Endpoint lista: method, path, request/response schema, példák.
56. Hibakódok és hibamodell (error object) egységesen.
57. Rate limit / abuse védelem elv (ha releváns).

58. Idempotencia és retry (ha releváns, pl. create műveleteknél).

Data model (docs/03_design/data_model.md)

Kötelező, ha van adat perzisztencia (DB, local storage, files, stb.).

Kötelező: Kötelező elemek:

59. Entitások (táblák/kollekciók) rövid leírással: cél, kulcs mezők.

60. Kapcsolatok és integritás (1:N, N:M, cascade).

61. Migráció / verziózás stratégia (hogyan változik a séma).

62. Adatélettartam: mit tárolsz, meddig, és hogyan törlöd (privacy miatt is fontos).

63. Indexelés vagy teljesítmény megfontolások (ha releváns).

Error handling (docs/03_design/error_handling.md)

Egységes hibakezelés nélkül nincs termékminőség. A cél, hogy a rendszer determinisztikusan viselkedjen hiba esetén, és a user tudja, mit tegyen.

Kötelező: Kötelező elemek:

64. Hibakategóriák: validációs hiba, auth hiba, not found, konfliktus, rate limit, belső hiba.

65. User-facing üzenetek elvei: érthető, cselekvésre ösztönöz (pl. 'Próbáld újra').

66. Retry stratégia: mikor retry, mikor nem (pl. network timeout vs. 400 validation).

67. Konzisztens error object (backend), és error mapping (frontend/mobil).

68. Logolás: mi kerül logba (de ne PII), és hogyan találod meg a hibát.

Elvárt bizonyítékok

- Legalább 1 contract jellegű teszt vagy OpenAPI validáció (ha van API).
- Legalább 2 hibahelyzet demonstrálása screenshot/log alapján (pl. invalid input, auth fail).
- Migráció demonstrálása (ha van DB): legalább 1 migration script, és hogy futtatod.

Minimum vs. kiemelkedő szint

- Minimum: dokumentált endpointok + hibamodell + egyszerű data model.
- Kiemelkedő: API verziózás és backwards compatibility gondolkodás; konfliktuskezelés (pl. optimistic locking).

Tipikus hibák

- API doksi csak felsorolás, nincs példa request/response.
- Hibaüzenetek keverednek (néha string, néha objektum), nincs standard.
- Adatmodell nincs frissítve a kódhoz képest.
- PII bekerül logba vagy hibába (privacy probléma).

Ajánlott AI-munkafolyamat

69. Kérd AI-tól OpenAPI vázlatot a user story-k alapján, majd implementáld és generáld újra az OpenAPI-t a valós kódból (ha lehet).

70. Kérd AI-tól standard error object javaslatot, majd írd bele a codebase-be és teszteld a tipikus hibákat.

71. Kérd AI-tól adatmodell normalizálási javaslatokat, de validáld a valós querykkel és használati mintákkal.

3.G Teszt stratégia + Teszt riport - bizonyítsd, hogy igaz

Cél

AI-val írt kódnál a tesztelés nem opcionális. A cél nem az, hogy minden sor le legyen fedve, hanem hogy a legfontosabb kockázatokra legyen regresszióvédelmed, és a rendszered determinisztikus legyen.

Teszt stratégia (docs/04_quality/test_strategy.md)

Elvárás: 1-2 oldal. Válaszolj a következő kérdésekre:

- 72. Mi a teszt piramisod? (unit, integration, e2e arány).
- 73. Mi az, amit unit tesztelsz (tisztán logika), és mi az, amit integrációval (DB, API)?
- 74. Mi az 1-2 legkritikusabb user flow, amit e2e teszt véd?
- 75. Milyen mock/stub stratégiát használsz, hogy a tesztek gyorsak és stabilak legyenek?
- 76. Milyen quality gate-ek vannak CI-ban? (pl. lint, format, teszt).
- 77. Hogyan futnak a tesztek lokálon és CI-ban (parancsok).

Minimum teszt mennyiség és összetétel

Minimum elvárás (platformfüggetlen):

- Legalább 30 automata teszt összesen.
- Legalább 18 unit jellegű teszt (core logika).
- Legalább 6 integrációs teszt (pl. DB + API együtt).
- Legalább 6 e2e/UI/contract jellegű teszt (a projektedhez illően).
- A tesztek futnak CI-ban (vagy legalább dokumentált módon futtathatók).

Teszt riport (docs/04_quality/test_report.md)

A teszt riport célja, hogy a reviewer 1 perc alatt lássa: mi fut, mi nem, és hol vannak kockázatok.

Kötelező: Kötelező tartalom:

- 78. Teszt suite-ek felsorolása (unit/integration/e2e) és futtatási parancsok.
- 79. Legutolsó futás dátuma és eredménye (pass/fail).
- 80. Ha van coverage: rövid összefoglaló (nem kötelező, de ajánlott).
- 81. Ismert hiányosságok: mi nincs letesztelve és miért (őszintén).
- 82. Flaky tesztek (ha vannak): hogyan kezeled.

Elvárt bizonyítékok

- CI log vagy screenshot, amin látszik a tesztek futása és eredménye.
- A repo-ban konkrét teszt fájlok, nem csak leírás.
- Ha performance állítás van: egy mérés/benchmark (akár egyszerű) és annak eredménye.

Minimum vs. kiemelkedő szint

- Minimum: 30 teszt + stratégia + riport, és futtatható parancsok.

- Kiemelkedő: contract teszt/OpenAPI validation, determinisztikus tesztdatok, és regressziós bug reprodukció tesztel.

Tipikus hibák

- Csak 'happy path' tesztek vannak, edge case-ek nélkül.
- Tesztek lokálon futnak, de CI-ban nem (vagy fordítva).
- Tesztek függnak külső API-tól és instabilak.
- Teszt riport üres vagy irreleváns (nincs futási eredmény).

Ajánlott AI-munkafolyamat

83. Kérd AI-tól: generáljon teszt eseteket a scope contract acceptance criteria alapján.
84. Implementáld a teszteseteket, majd futtasd őket többször: stabilak-e? (flaky detektálás).
85. Ha egy teszt hibás, ne csak 'javítsd', hanem írd be a verification log-ba: miért volt rossz az AI javaslat.
86. Kérd AI-tól: javasoljon quality gate-eket CI-ra (lint, format, test), majd állítsd be.

3.H Security + Privacy + Licensing - felelős termék

Cél

Egy termék nem csak 'működik', hanem biztonságos, és felelősen bánik az adatokkal. A security és privacy dokumentáció nem jogi traktátus, hanem mérnöki kockázatkezelés: mit támadhatnak, milyen adatot kezelsz, és hogyan csökkentet a kockázatot.

Threat model (docs/05_security_ops/threat_model.md)

Minimum elvárás: STRIDE jellegű áttekintés legalább 6 fenyegetéssel (nem kell mind a 6 kategória, de legyen változatos).

Kötelező: Kötelező tartalom:

87. Attack surface: milyen belépési pontok vannak (UI, API, admin, integráció).
88. Fenyegetések listája (legalább 6): rövid leírás, hatás, valószínűség.
89. Mitigáció: mit csinálsz ellene (kód, config, infra).
90. Residual risk: mi marad meg és miért fogadod el.
91. Verification: hogyan ellenőrzöd (teszt, linter, dependency scan, manuális ellenőrzés).

Privacy + adatkezelés (docs/05_security_ops/privacy_licensing.md)

Kötelező, akkor is, ha csak minimális adatot kezelsz. Legalább fogalmazd meg: mit tárolsz és miért.

Kötelező: Kötelező elemek:

92. Adatkategóriák: milyen adatokat kezelsz (PII? email? location? logok?).
93. Adatáramlás: honnan jön adat, hol tárolod, hova küldöd (külső szolgáltatás?).
94. Adatmegőrzés és törlés: mennyi ideig, hogyan törölhető.
95. Hozzáférés: ki látja (role-ok), admin funkciók vannak-e.
96. AI használat és adatok: küldesz-e PII-t az AI-nak? (elvárás: ne küldj).
97. Harmadik fél függőségek és licenc: főbb libek/SDK-k listája, és hogy kompatibilisek-e.

Kötelező security baseline a kódban

- Nincs hardcoded secret. Minden titok környezeti változóban vagy secrets store-ban van.
- Input validáció (backend és/vagy UI).
- Auth és authorization: ha van user rendszer, a védett endpointokat védj ténylegesen.
- Error handling: ne szivároogjon stack trace/PII a kliens felé.
- Dependency hygiene: ne legyen ismert kritikus sérülékenység (ha van scan eszköz, futtasd).

Elvárt bizonyítékok

- Threat model táblázat kitöltve + mitigációk a kódban visszaköthetők (link).
- Secrets kezelés megoldása (.env.example, CI secrets) dokumentálva.
- Legalább 1 security jellegű teszt vagy ellenőrzés (pl. auth test, input validation test).

Minimum vs. kiemelkedő szint

- Minimum: threat model + adatkezelés leírás + secrets hygiene.
- Kiemelkedő: alap dependency scan vagy SAST/linters futtatása CI-ban, és konkrét bizonyíték a javításról.

Tipikus hibák

- Privacy leírás nincs, 'nem kezelek adatot' - közben logol PII-t.
- Mitigáció csak szövegben van, kódban nincs nyoma.
- Auth csak UI oldalon van (backend nem védi).
- Secret bekerül a repo-ba (pl. config fájlban).

Ajánlott AI-munkafolyamat

98. Kérd AI-tól: készítsen STRIDE threat listát a rendszered architektúrája alapján.
99. Válassz 6-8 releváns fenyegetést, és írd mellé a valós mitigációt (amit tényleg implementálsz).
100. Kérd AI-tól: ellenőrzőlistát PII/log/secret témában, majd kézzel auditáld a kódot.

3.1 Deploy + Runbook + Observability - üzemeltethető rendszer

Cél

A termékminőség része, hogy a rendszert nem csak fejlesztői gépen lehet futtatni, hanem reprodukálható módon telepíthető, konfigurálható, és hiba esetén diagnosztizálható. Nem kell 'enterprise' üzemeltetés, de legyen világos: hogyan lesz ebből működő szolgáltatás.

Deploy runbook (docs/05_security_ops/deploy_runbook.md)

Kötelező: Kötelező elemek:

101. Környezeti modell: lokál / staging / prod (ha csak lokál van, akkor is írd le).
102. Deploy lépések: parancsok, sorrend, előfeltételek (pl. DB migráció).
103. Konfiguráció és secrets: honnan jön, hogyan állítod be.
104. Rollback: ha deploy után rossz, mit csinálsz (1-2 lépés).
105. Verziózás: hogyan azonosítod a release-t (git tag, build szám).

Runbook (operáció) - minimum incident forgatókönyvek

A runbook célja: ha 02:00-kor elromlik, mit csinálsz. Minimum 2 incident scenariót írnak le.

Példák: Kötelező incident példák (válassz legalább kettőt):

- API nem válaszol / 500-as hibák tömegesen.
- Adatbázis kapcsolat megszakad.
- Külső API limit vagy kiesés.
- Felhasználók nem tudnak bejelentkezni.
- Disk space/ memória elfogy.

Kötelező: Minden incident scenario-hoz írd le:

- Tünetek (mit lát a user / log).
- Gyors diagnózis lépések (mit nézzek először).
- Ideiglenes mitigáció (pl. feature flag, cache flush).
- Végleges javítás iránya (milyen ticket keletkezik).

Observability (docs/05_security_ops/observability.md)

Kötelező: Kötelező elemek:

106. Logging: milyen log szintek (info/warn/error), struktúra (pl. request id), és mit nem logolsz (PII).
107. Healthcheck: van-e endpoint / check, és mit vizsgál (DB, külső API).
108. Metrikák: legalább 3 javasolt metrika (latency, error rate, throughput, queue length).
109. Tracing (opcionális): ha van, hogyan követed a requesteket.
110. Debugging guide: hol találok a logokat, mit keressek.

Elvárt bizonyítékok

- Deploy lépések ténylegesen működnek (screenshot vagy CI run).
- Healthcheck válasz screenshot (pl. /health).
- Log minta (1-2 sor) a docs-ban, ami egy hiba esetet mutat.

Minimum vs. kiemelkedő szint

- Minimum: run parancs + deploy runbook + 2 incident scenario + healthcheck/log alapok.
- Kiemelkedő: egyszerű monitoring dashboard koncepció (nem kell grafana, de legyen terv), és SLO gondolkodás (pl. 99% uptime cél).

Tipikus hibák

- Deploy leírás csak annyi: 'feltöltöttem', nincs konkrét lépés.
- Runbook üres, vagy csak általános tanácsok (nincs konkrét log hely).
- Logolás túl sok vagy túl kevés (nincs request azonosító, nincs kontextus).

Ajánlott AI-munkafolyamat

111. Kérd AI-tól: készítsen deploy runbook vázlatot a stack alapján (Docker, cloud, stb.), majd igazítsd a valós projecthez.

112. Kérd AI-tól: javasoljon 5 tipikus incidentet a rendszeredhez, majd válassz kettőt és írd le konkrét lépésekkel.
113. Kérd AI-tól: strukturált log formátumot (json fields), majd implementáld és mutass példát.

3.J AI dokumentáció - átlátható és verifikált AI-first fejlesztés

Cél

Mivel a teljes fejlesztés AI-val történik, az értékelés egyik kulcsa: hogyan használd az AI-t mérnöki módon. A 'használtam AI-t' nem elég. A cél: nyomon követhető legyen, mit generált az AI, és azt hogyan ellenőrizted. Ez védi a projektet a hallucinációktól, licenc/kódminőség problémáktól és rejtett hibáktól.

Kötelező AI artefaktok

114. AI Manifest (docs/07_ai/ai_manifest.md)
115. Prompt Log (docs/07_ai/prompt_log.md)
116. Verification Log (docs/07_ai/verification_log.md)

AI Manifest - tartalom és elvárás

Cél: 1-2 oldal. Írd le, milyen AI eszközöket használtál, mire, és milyen korlátokkal.

Kötelező: Kötelező mezők:

117. Használt eszközök és verziók (pl. ChatGPT, Copilot, Cursor, stb.).
118. Felhasználási területek: tervezés, kód, teszt, doku, debug.
119. Tiltások: milyen adatot nem adtál ki az AI-nak (PII, secret).
120. Kritikus döntések: 3-5 olyan pont, ahol te döntöttél (nem az AI), és miért.
121. Kockázatok: hallucináció, hibás security tanács, licenc - és hogyan kezeled.

Prompt Log - tartalom és forma

Cél: 10-20 legfontosabb prompt és a kimenet rövid kivonata. Nem kell minden prompt, csak a lényeges döntéseket befolyásoló vagy nagyobb kódot generáló promptok.

Kötelező: Kötelező mezők minden prompt bejegyzésnél:

- Dátum / iteráció (pl. P-07).
- Cél (mit akartál elérni).
- Prompt szöveg (rövidítve oké, de legyen érthető).
- AI válasz rövid kivonata (mit javasolt).
- Hová került az eredmény (fájl/link).
- Mit módosítottál kézzel (ha volt).

Verification Log - a legfontosabb rész

Cél: bizonyítsd, hogy az AI által javasolt megoldások igazak és működnek. Minimum 5-10 verifikációs bejegyzés.

Kötelező: Kötelező mezők minden verifikációs bejegyzésnél:

122. AI állítás / javaslat (pl. 'ez a regex jó email validációra', 'JWT így biztonságos').

- 123. Kockázat: mi a baj, ha nem igaz (bug, security issue, data loss).
- 124. Ellenőrzés módja: teszt, mérés, forrás, PoC, code review checklist.
- 125. Eredmény: pass/fail + konkrét output (log, screenshot, teszt eredmény).
- 126. Következtetés: mi változott a kódban/dokuban a verifikáció után.

AI használat minőségi szabályai (kötelező betartani)

- Ne bízz az AI-ban vakon: minden security/performance/jogi állítást ellenőrizni kell.
- Promptokban ne szerepeljenek titkok és személyes adatok.
- A generált kódot refaktoráld: naming, modulhatárok, error handling, tesztelhetőség.
- A generált dokumentációt is validáld: ne írjon olyat, ami nincs implementálva.
- Ha az AI hibázott, dokumentáld a verification logban (ez nem büntetés, hanem mérnöki érettség).

Elvart bizonyítékok

- AI manifest + prompt log + verification log kitöltve a repo-ban.
- Legalább 2 olyan verifikáció, ami tesztet is eredményezett (pl. security vagy edge case).
- Legalább 1 olyan verifikáció, ami mérés (pl. latency) vagy PoC (pl. migration) alapján történt.

Minimum vs. kiemelkedő szint

- Minimum: 10-20 prompt log + 5 verifikációs bejegyzés.
- Kiemelkedő: verifikációk explicit kockázat-alapúak, és közvetlenül jobb tesztek/architektúrát eredményeznek.

4. Értékelés és pontozás

4.1 Kapu feltételek (ha nincs meg, plafon vagy levonás)

A kapu feltételek célja, hogy a termék alap minőségi minimumai teljesüljenek. Ha valamelyik kapu feltétel nem teljesül, az értékelés plafonos vagy a leadás javításig nem elfogadható.

Kapu feltétel	Elvárás	Ha hiányzik	Bizonyíték
Futtathatóság	README alapján 15 percen belül elindítható a rendszer.	Max. 40 pont adható.	Quickstart + futó rendszer screenshot/log
AI átláthatóság	AI manifest + prompt log + verification log megvan.	Max. 70 pont adható.	docs/07_ai/*
Secret hygiene	Nincs repo-ban token/jelszó/API kulcs.	Leadás javításig nem elfogadható, vagy -20 pont.	.env.example + repo audit
Automata tesztek	Legalább 30 automata teszt, futtatható	Max. 70 pont adható.	CI log + teszt fájlok

	módon.		
--	--------	--	--

4.2 Pontozási rubrika (100 pont)

Az értékelés célja, hogy a különböző témájú projektek is összehasonlíthatóak legyenek. Ezért a rubrika a termékminőség dimenzióira pontoz, nem a kód mennyiségére.

Minden kategóriánál: (a) mit nézünk, (b) mi számít minimumnak, (c) mi számít kiemelkedőnek, (d) milyen evidence kell.

Kategória	Pont	Mit ellenőrzünk (röviden)	Evidence (példák)
A) Product & Scope	12	Vision, persona, MVP, non-goals, acceptance criteria, metrikák.	vision.md, scope_contract.md, metrics.md
B) Capability Breadth	10	Legalább 6 capability, value + productization arány, evidence + teszt.	capability_map.md linkekkel
C) Architecture & Decisions	13	C4, quality attributes, ADR-ek, tradeoffok.	c4_*.md, adr/*.md
D) Engineering Quality	15	Modularitás, hibakezelés, konfigurálhatóság, technikai adósság kezelése.	kód + error_handling.md
E) Testing & Quality Gates	15	Teszt stratégia, 10 teszt, CI quality gate, regresszióvédelem.	test_strategy.md + CI log
F) DevOps & Operations	15	Deploy runbook, rollback, runbook, healthcheck, logolás.	deploy_runbook.md, observability.md
G) Security, Privacy, Licensing	10	Threat model, adatkezelés, secrets, auth ellenőrzés.	threat_model.md, privacy_licensing.md
H) AI Engineering Maturity	10	Promptok + verifikációk minősége, kockázat-alapú ellenőrzés.	ai_manifest.md, verification_log.md

4.3 Részletes értékelési leírás kategóriánként

A) Product & Scope (12 pont)

Mit nézünk: termék-szemlélet, világos cél és scope, objektív elfogadási kritériumok, mérhetőség.

Minimum (0-40%):

- Van vision, de általános; persona hiányos vagy túl tág.
- MVP lista nincs prioritizálva vagy túl nagy.
- Elfogadási kritériumok hiányoznak vagy nem tesztelhetők.

Jó (40-80%):

- 1-2 persona, világos probléma és értékígéret.
- 3-6 MVP story, mindegyikhez acceptance criteria.
- Non-goals listázva; scope fegyelem látszik.

Kiemelkedő (80-100%):

- Metrikák (North Star + guardrail) és mérési terv reális.
- Kockázatok és mitigációk konkrétak, és visszaköthetők a megvalósításhoz.
- A scope változásai dokumentáltak (changelog/ADR).

Elvárt evidence:

- vision.md, scope_contract.md
- metrics.md (ha van külön)
- release changelog vagy commit history

B) Capability Breadth (10 pont)

Mit nézünk: a termék mennyire széles képességű; van-e productization (nemfunkcionális) lefedés.

Minimum (0-40%):

- Kevesebb mint 6 capability, vagy mind value jellegű.
- Done capability-khez nincs teszt vagy evidence.
- A capability megnevezések technikaiak, nem képesség nyelvűek.

Jó (40-80%):

- 6-8 capability, legalább 3 value és 3 productization.
- Minden Done-hoz evidence + teszt link.
- A map alapján érthető a termék.

Kiemelkedő (80-100%):

- Capability-k lefedik a termékminőség több dimenzióját (auth, operálhatóság, hibakezelés, CI).
- Státusz és roadmap (mi planned és miért).
- Képességekhez mérési ötlet vagy kockázat jelölés.

Elvárt evidence:

- capability_map.md
- screenshotok/GIF-ek a docs-ban
- CI log + tesztek linkjei

C) Architecture & Decisions (13 pont)

Mit nézünk: van-e érthető architektúra, dokumentált döntések, és minőségi attribútum gondolkodás.

Minimum (0-40%):

- Nincs C4, vagy csak 1 homályos diagram.
- ADR-ek hiányoznak vagy csak a döntés van, alternatíva nélkül.
- Quality attributes csak buzzword lista.

Jó (40-80%):

- Context + Container diagram, olvasható és pontos.
- Legalább 5 ADR tradeoffokkal.
- 5-8 quality attribute, legalább 1 scenario formában.

Kiemelkedő (80-100%):

- Component szint (ha indokolt) és deployment view is van.
- Legalább 2 quality attribute scenario mérőszámokkal.
- ADR-ekben verification és evidence (benchmark/PoC/test).

Elvárt evidence:

- c4_context_container.md, c4_component.md (ha van)
- quality_attributes.md
- adr/*.md

D) Engineering Quality (15 pont)

Mit nézünk: kódminőség, karbantarthatóság, hibakezelés, konfigurálhatóság, alap performance gondolkodás.

Minimum (0-40%):

- Spagetti szerkezet, duplikáció, névadás következetlen.
- Hibakezelés ad-hoc; nincs standard.
- Konfiguráció hardcoded.

Jó (40-80%):

- Moduláris szerkezet, következetes naming, separation of concerns.
- Egységes hibamodell (error object vagy standard exception).
- Konfiguráció env-alapú, dokumentált.

Kiemelkedő (80-100%):

- Egységes logging, request-id/trace-id gondolkodás.
- Tech debt lista + rövid roadmap (mit csinálnál még 10 órában).
- Performance alap ellenőrzés (mérés vagy benchmark) ha releváns.

Elvárt evidence:

- kód (strukturált), error_handling.md
- log minták / observability.md
- performance.md (ha van)

E) Testing & Quality Gates (15 pont)

Mit nézünk: regresszióvédelem, tesztek minősége, CI quality gate-ek.

Minimum (0-40%):

- Nincs vagy kevés teszt (<30).
- Tesztek nem futnak CI-ban vagy instabilak.
- Nincs teszt stratégia.

Jó (40-80%):

- 30+ teszt, mix: unit + integration + e2e/contract.
- CI futtatja a teszteket.
- Teszt riportban látszik a legutolsó futás eredménye.

Kiemelkedő (80-100%):

- Contract/OpenAPI validation vagy snapshot tesztek.
- Flaky teszt kezelés, determinisztikus tesztadatok.
- Kritikus bug reprodukció teszttel (post-mortem jelleg).

Elvárt evidence:

- test_strategy.md, test_report.md
- CI futás link/screenshot
- teszt fájlokra mutató linkek

F) DevOps & Operations (15 pont)

Mit nézünk: deployálhatóság, üzemeltethetőség, runbook, observability alapok.

Minimum (0-40%):

- Csak lokálon megy, nincs deploy/runbook.
- Nincs healthcheck vagy logolás.
- Rollback nincs leírva.

Jó (40-80%):

- Deploy runbook lépésről lépésre, konfigurációval és rollbackkel.
- Legalább 2 incident scenario a runbookban.

- Healthcheck + alap logolás dokumentált.

Kiemelkedő (80-100%):

- Környezetek (staging/prod) koncepció, release flow, verziózás.
- Metrikák és SLO gondolkodás (targetek).
- Megfigyelhetőség: strukturált log, trace id, hibák gyors diagnózisa.

Elvárt evidence:

- deploy_runbook.md, observability.md
- health endpoint screenshot/log
- CI deploy step (ha van) vagy docker compose

G) Security, Privacy, Licensing (10 pont)

Mit nézünk: fenyegetések felismerése, mitigációk, adatkezelés felelősen, licenc alapok.

Minimum (0-40%):

- Threat model hiányzik vagy túl általános.
- Privacy leírás hiányzik.
- Secret bekerül a repo-ba.

Jó (40-80%):

- 6+ fenyegetés, mitigációkkal, és legalább 1 verification.
- Adatkategóriák és adatáramlás leírva.
- Secret hygiene rendben, auth védelem valós.

Kiemelkedő (80-100%):

- Dependency scan/SAST vagy hasonló ellenőrzés CI-ban.
- Mitigációk bizonyítottan működnek (teszt).
- Residual risk őszintén dokumentált.

Elvárt evidence:

- threat_model.md, privacy_licensing.md
- .env.example, CI secrets
- auth/input validation tesztek

H) AI Engineering Maturity (10 pont)

Mit nézünk: AI használat minősége, verifikáció, reprodukálhatóság, tanulságok.

Minimum (0-40%):

- Csak annyi van: 'AI-t használtam'.
- Prompt log hiányos vagy nem köthető artefaktokhoz.
- Verifikáció nincs vagy csak szöveg, evidence nélkül.

Jó (40-80%):

- 10-20 fontos prompt dokumentálva fájl linkekkel.
- 5-10 verifikációs bejegyzés kockázattal és eredménnyel.
- AI manifest összege, tiltásokkal (PII/secrets).

Kiemelkedő (80-100%):

- Verifikációk konkrétan jobb tesztek/architektúrát eredményeztek.
- AI hibák és korlátok dokumentálva (tanulságok).
- Reprodukálható: ugyanaz a prompt -> hasonló eredmény, és beépített ellenőrzések.

Elvárt evidence:

- ai_manifest.md, prompt_log.md, verification_log.md
- teszt commitok, benchmark/PoC evidence

4.4 Bónusz (0-10 pont, opcionális)

A bónusz célja, hogy a nagyon érett megoldások extra elismerést kapjanak. Nem kötelező a 600 órába.

- +2: Feature flag vagy konfigurálható funkciókapcsoló (release kontroll).
- +2: Offline/queue/retry stratégia (mobilnál különösen érték).
- +2: Accessibility vagy i18n extra (ténylegesen kipróbálható).
- +2: Rate limiting / abuse prevention (API-nál).
- +2: Rövid post-mortem egy valós hibáról: ok, hatás, javítás, preventív teszt.

4.5 Önértékelés (kötelező melléklet a leadásban)

A hallgató a leadásban készít egy rövid önértékelést: kategóriánként mennyi pontot adna magának és miért. Ez segít abban, hogy értse, mivel kell még foglalkoznia a termékminőséghez.

Kategória	Önpont (0- max)	Indoklás (1-2 mondat)	Mit csinálnék még +10 órában?
A			
B			
C			
D			
E			
F			
G			
H			
Bónusz			

5. Végállapot: leadási elvárások

Ez a fejezet a LEADÁS VÉGÉRE elvárt állapotot írja le. Nem a fejlesztési folyamatot értékeli, hanem azt, hogy a kész termék és a repo-dokumentáció együtt termékminőségű-e.

A leadandó akkor tekinthető release-közelinek, ha egy külső fejlesztő/üzemeltető a repo alapján képes: (1) elindítani, (2) megérteni, (3) validálni a minőséget (teszt/CI), (4) biztonságosan konfigurálni, (5) diagnosztizálni a hibákat, (6) kiadni és visszagörgetni, (7) követni az AI-val készült döntések és kódrészletek verifikációját.

5.1 Leadási csomag definíciója

A leadandó csomag egyetlen Git repo. A repo a kódot, a dokumentációt és a bizonyítékokat (evidence) együtt tartalmazza. A leadásban NINCS külön, e-mailben küldött „pótdoksi” - minden linkelhető és verziózott.

A repo-ban kötelezően legyen:

- README.md mint belépési pont (mi ez, kinek szól, quickstart, tesztek, demo).
- docs/ könyvtár a teljes dokumentációval (Docs-as-Code, verziózva).
- Reprodukálható futtatás: docker compose / lokál build leírás / deploy leírás (projektfüggő).
- Automata tesztek és futtatási mód (lokál + CI).
- CI pipeline definíció (build, lint/format, teszt, security alap ellenőrzések).
- Konfiguráció/secret kezelés dokumentálva (nincs titok a repo-ban).
- Demo script és release/changelog (mit tud a termék és hogyan mutatód be stabilan).
- AI dokumentáció: AI manifest + prompt log + verification log.

5.2 Release-ready elfogadási kritériumok

Az alábbi kritériumok a „release candidate” jellegű, termékminőségű leadás minimumai. Ha valamelyik pont nem teljesül, akkor a leadás jellemzően demo-szintű marad, és a pontszám plafonos lehet a 4.1 kapu feltételek alapján.

- [] A README quickstart alapján egy külső ember 15-30 perc alatt elindítja az alkalmazást (lokál vagy demo környezet).
- [] A main/default branch-en a CI zöld: build + lint/format + tesztek lefutnak; a pipeline log linkelhető.
- [] Nincs repo-ban hardcoded secret (API kulcs, jelszó, token). Secret kezelés módja dokumentált.
- [] Van minimum értelmes tesztmix: unit + integráció + end-to-end/contract jellegű tesztek; nem csak „happy path”.
- [] A dokumentáció összhangban van a kóddal (nincs elavult diagram / hamis állítás).
- [] A hibakezelés és edge case-ek demonstrálhatók (error state UX és backend error modell).
- [] Van release/runbook szemlélet: deploy lépések, konfiguráció, rollback, tipikus hibák és teendők.
- [] Van alap security és privacy gondolkodás (threat model, adatkezelés, jogosultságok).
- [] AI használat nyomon követhető: promptok + verifikációk; a kritikus állítások tesztel/méréssel igazoltak.

5.3 Technikai mélység elvárások területenként

Az alábbi alfejezetek azt részletezik, hogy a végállapotban milyen technikai mélységet várunk el. Minden pontnál a legfontosabb elv: állítás helyett bizonyíték. Ha valami a projektedben nem releváns, akkor jelöld „N/A”-ként, írd le röviden miért, és adj helyette egy hasonló értékű alternatív minőségi artefaktot (pl. contract teszt helyett modulhatár integrációs teszt).

5.3.1 Futtathatóság, build és konfiguráció

Cél: a projekted ne „csak a laptopodon” működjön. A futtatás legyen determinisztikus, dokumentált és hibakereshető.

Elvárt a végére:

- Dokumentált előfeltételek (SDK verziók, DB, build tool, mobilnál Xcode/Android Studio verzió).
- Quickstart lépések pontos parancsokkal (copy-paste), és a várható kimenet rövid leírásával.
- Konfiguráció elkülönítve (env/config fájl), és van példa konfiguráció: ``.env.example` / `config.example.*``.
- Seed/demo adatok vagy teszt user(ek) biztosítva (hogy az értékelő ne üres képernyőt kapjon).
- Hibák esetén a rendszer értelmes üzenetet ad (nem stack trace a usernek; logban viszont látszik a hiba oka).
- Build folyamat verziózott és reprodukálható (Dockerfile vagy build script vagy dokumentált IDE build).

Erősen ajánlott (jobb pontszám / érettebb termék):

- Egyparancsos indítás: ``make up` / `docker compose up` / `npm run dev`` jelleggel (projektfüggő).
- Külön dev és prod konfiguráció (legalább elvi szinten), és dokumentált az eltérés.
- Konfiguráció validáció induláskor (hiányzó env var -> fail fast, érthető hiba).
- Container image vagy build artifact verziózása (tagging: commit SHA/semver).

Elfogadott evidence példák:

- Képernyőkép vagy log arról, hogy „szűz” környezetben fut (pl. friss container build).
- CI run link, ahol a build + teszt lefutott.
- ``.env.example`` és README részlet, ami működik.

5.3.2 Funkcionalitás és UX

Cél: a termék ne csak funkciókat tartalmazzon, hanem használható, koherens élményt adjon, edge case-ekkel együtt.

Elvárt a végére:

- Scope Contract szerinti MVP teljesül, és minden user story-hoz van elfogadási kritérium.
- Legalább 2-3 fő user flow dokumentált (happy path + tipikus hiba/empty állapot).
- Állapotkezelés konzisztens (loading, empty, error, success állapotok nincsenek „kifelejtve”).
- Input validáció és hibák: a user értelmes visszajelzést kap (nem „valami hiba történt”).

- Alap hozzáférhetőség (a11y): olvashatóság/kontraszt, fókusz/billentyűzet vagy mobil platform guideline-ok minimuma.

Erősen ajánlott (jobb pontszám / érettebb termék):

- Onboarding vagy első használat támogatása (üres állapot magyarázó szöveg, minták).
- Instrumentált UX: legalább elvi mérőszámok (pl. funnel lépések, sikeres műveletek aránya).
- Mobilnál: offline tolerancia (cache) vagy hálózati hibák jó kezelése; webnél: responszív layout minimum.

Elfogadott evidence példák:

- Képernyőképek/gif-ek a fő flow-król.
- Demo script, ami NEM improvizáció (docs/06_release/demo_script.md).
- Elfogadási kritériumok linkje a scope_contract.md-ben.

5.3.3 Architektúra, modulhatárok és bővíthetőség

Cél: a rendszered legyen érthető, indokolható és bővíthető. 600 óránál elvárás, hogy a struktúra tudatos legyen.

Elvárt a végére:

- C4 Context + Container diagram naprakészen, a valós kódhoz igazítva.
- Komponens/modul szintű bontás (C4 Component vagy részletes leírás), felelőségekkel.
- Minimum 8 ADR a kritikus döntésekhez (miért így? alternatívák? következmények?).
- Kód-szintű modulhatárok: külön rétegek / modulok (UI, domain, data, infra) vagy más tudatos felosztás.
- Konfiguráció, logging, error handling, auth ne legyen szanaszét másolva; legyen közös megoldás/pattern.

Erősen ajánlott (jobb pontszám / érettebb termék):

- Minőségattribútumok és konkrét quality scenario-k (pl. p95 latency cél, availability, maintainability).
- API versioning / backwards compatibility terv (ha publikus API jellegű).
- Tech debt lista: mi maradt kompromisszum és miért; roadmap 3-5 pont.

Elfogadott evidence példák:

- Diagramok a docs/02_architecture alatt.
- ADR fájlok linkelve a README-ből vagy indexből.
- Kódban látható moduláris szerkezet (mappák, layer boundaries).

5.3.4 Adatmodell és API szerződés

Cél: az adatok és interfészek legyenek versionálhatók, validáltak és dokumentáltak.

Elvárt a végére:

- Adatmodell dokumentálva (entitások, mezők, kapcsolatok, invariánsok).
- Migráció/seed stratégia (ha van DB), vagy helyettesítő verziózás (pl. local storage schema version).

- API szerződés dokumentálva (OpenAPI vagy részletes endpoint leírás), hibamoddell.
- Input/output validáció (server oldalon és/vagy kliens oldalon), és konzisztens hibakódok.
- Idempotencia és retry viselkedés dokumentálva ott, ahol releváns (pl. fizetés, rendelés, submit).

Erősen ajánlott (jobb pontszám / érettebb termék):

- Contract tesztek (consumer/provider vagy snapshot) az API stabilitására.
- DB indexelés / query optimalizálás legalább egy kritikus útvonalra (mérés + javítás).
- Séma változás hatásának kezelése (backward compatible migrációk).

Elfogadott evidence példák:

- OpenAPI export vagy linkelt swagger UI.
- Migráció fájlok / schema versioning dokumentáció.
- Integrációs tesztek, amelyek valós DB-vel futnak (container).

5.3.5 Tesztelés és regresszióvédelem

Cél: az AI-val generált kódnál a teszt a bizalom alapja. Nem a tesztek száma a lényeg, hanem hogy a kockázatos részek védve vannak.

Elvárt a végére:

- Teszt stratégia: mit és miért unit/integration/e2e szinten (docs/04_quality/test_strategy.md).
- Minimum 30 automata teszt összesen, értelmes megoszlással (ajánlás: 18 unit, 6 integráció, 6 e2e/contract - projekthez igazítva).
- CI-ban futó tesztek: a main branch-re csak zöld állapot mehet (merge/push policy projekttől függően).
- Legalább 5 negatív teszt (hibák, invalid input, jogosultság, hálózati hiba, üres adat).
- Teszt riport: mi fut, hol fut, mi a lefedettség és a hiány (docs/04_quality/test_report.md).

Erősen ajánlott (jobb pontszám / érettebb termék):

- 60+ teszt, kockázat-alapú bővítéssel (kritikus flow-k és edge case-ek erős védelme).
- Contract teszt vagy API snapshot (ha API-s rendszer).
- Performance smoke teszt (pl. 1-2 kritikus endpoint p95 mérése) vagy mobilnál indítási idő / memory baseline.
- Flaky teszt menedzsment: ismert flake-ek listája és stabilizálási terv.

Elfogadott evidence példák:

- CI teszt run link + összefoglaló (badge vagy screenshot).
- Teszt fájlok linkje + rövid magyarázat a test_strategy.md-ben.
- Negatív tesztek konkrét példái (fájlnév + tesztnevé).

5.3.6 CI/CD, kiadás és üzemeltethetőség

Cél: a termék kiadható legyen, ne csak futtatható. A pipeline és a runbook a termék része.

Elvárt a végére:

- CI pipeline legalább: build + lint/format + teszt; hibára megáll.
- Build artifact vagy container image előállítás (ahogy a projekt megköveteli).
- Deploy leírás: hogyan kerül éles/demó környezetbe (lépések, konfiguráció, jogosultságok).
- Rollback terv: mit csinálsz, ha rossz release ment ki (minimum leírva, ideális esetben kipróbálva).
- Runbook: 2 tipikus incident forgatókönyv (pl. DB nem elérhető, külső API down) és teendők.

Erősen ajánlott (jobb pontszám / érettebb termék):

- Staging környezet (akár egyszerű) a prod-szerű validációhoz.
- Release verziózás (semver vagy commit SHA) és changelog vezetése.
- Dependency update stratégia (renovate/dependabot vagy manuális policy).
- Feature flag vagy konfigurációs kapcsolók (bónuszhoz is kapcsolódik).

Elfogadott evidence példák:

- CI workflow fájl(ok) + egy sikeres run link.
- Deploy script / infra definíció / hosting beállítás leírás (projektfüggetlen).
- Runbook dokumentum linkje a README-ből.

5.3.7 Megfigyelhetőség és diagnosztika

Cél: ha baj van, tudd megmondani: mi romlott el, hol, miért, és mennyi felhasználót érint.

Elvárt a végére:

- Strukturált logging (kulcs-érték vagy JSON) legalább a kritikus műveleteknél.
- Kérés/folyamat korreláció (request_id/correlation_id) ahol értelmezhető.
- Healthcheck (liveness/readiness) vagy platformhoz illő megfelelő (mobilnál belső diagnosztikai képernyő is lehet).
- Alap metrikák (min. 3): pl. hibaarány, latency (p95), sikeres műveletek száma; definícióval.
- Logolási szabály: PII nem kerül logba (vagy anonimizálva).

Erősen ajánlott (jobb pontszám / érettebb termék):

- Tracing (OpenTelemetry vagy analóg) és legalább 1 end-to-end trace bemutatása (screenshot).
- Alert alapok: 1-2 riasztási szabály és mit tennél rá (runbook link).
- Kliens oldali crash/exception gyűjtés (mobil/web): legalább koncepcionálisan dokumentálva.

Elfogadott evidence példák:

- Log minta (1-2 sor) a docs/05_security_ops/observability.md-ben.
- Metrikák definíciója + screenshot (ha van dashboard).
- Health endpoint vagy diagnosztikai képernyő bemutatása.

5.3.8 Biztonság, adatvédelem és licenc

Cél: a termék ne legyen vállalhatatlan kockázat. A minimum security baseline kötelező.

Elvárt a végére:

- Threat model (STRIDE jelleggel) minimum 6 konkrét fenyegetéssel és mitigációval.

- Secret hygiene: semmi titok a repo-ban; env/secret store használat; rotáció elv leírva.
- AuthN/AuthZ (ha releváns): legalább egy világos jogosultsági modell (pl. user/admin, RBAC), és tesztelve.
- Dependency kockázatok: függőségek listája és alap sérülékenységi-ellenőrzés (tool + eredmény).
- Privacy: kezelt adatok listája (PII?), adatmegőrzés/törlés elv, és milyen adat megy az AI eszközökbe.

Erősen ajánlott (jobb pontszám / érettebb termék):

- OWASP Top 10 releváns tételek rövid kezelése (input validation, auth, access control, logging).
- Security tesztek: legalább 2-3 célzott teszt (pl. jogosultság megtagadás, injection jellegű input).
- Rate limiting / abuse prevention (API-nál) vagy mobilnál: secure storage, certificate pinning koncepció.

Elfogadott evidence példák:

- Threat model táblázat + mitigációk linkje (kód/konfig).
- Dependabot/renovate report vagy `npm audit`/`pip-audit`/`gradle` riport.
- Secret scanning eredmény (ha használ sz ilyen) vagy explicit audit jegyzet.

5.3.9 AI engineering és verifikáció

Cél: az AI használat ne csak gyors legyen, hanem mérnökiileg kontrollált. A verifikáció hiánya termékminőségi kockázat.

Elvárt a végére:

- AI manifest: milyen eszközöket használtál és mire (kód, teszt, doku, design).
- Prompt log: a 10-20 legfontosabb prompt kontextussal + melyik artefakt készült belőle.
- Verification log: legalább 10 tétel, ahol AI állítást ellenőriztél (teszt, mérés, forrás, PoC).
- Kritikus döntéseknél (auth, data, security, architektúra) legyen explicit „miért” és verifikáció.
- AI által generált kódnál: minimálisan futtatott teszt + rövid „review checklist” (mit ellenőriztél).

Erősen ajánlott (jobb pontszám / érettebb termék):

- AI kimenetek értékelése: hibák és korlátok dokumentálva (tanulságok).
- Promptok reprodukálhatósága (verziózott kontextus, input fájlok).
- Automatikus minőségi kapuk AI kódra (lint, static analysis, typecheck) CI-ban.

Elfogadott evidence példák:

- docs/07_ai/ai_manifest.md, prompt_log.md, verification_log.md linkelve.
- Konkrét példák: AI állítás -> teszt commit -> zöld CI.

6. Önellenőrzés: készultság és termékminőség mérése

Ezt a fejezetet bármikor használhatod: megmutatja, hogy MOST mennyire vagy kész, és mi hiányzik ahhoz, hogy a leadás release-közeli, termékminőségű legyen. A kitöltött scorecard a leadás kötelező melléklete (4.5).

[]/[x]	Ellenőrzés	Hol találom? (link/fájl)
[]	A repo klónozás után a README Quickstart alapján 15 percen belül elindul.	README.md
[]	Nincs a repo-ban secret (token/jelszó). Van .env.example.	.env.example + audit
[]	Van docs index és minden fejezet kitöltött (nincs TODO).	docs/00_index.md
[]	Vision + Scope Contract kész, acceptance criteria-val.	docs/01_product/*
[]	Capability map 6+ képességgel, evidence + teszt linkkel a Done-okra.	docs/01_product/capability_map.md
[]	C4 Context + Container diagram megvan.	docs/02_architecture/*
[]	Van legalább 5 ADR (alternatívák + tradeoffok).	docs/02_architecture/adr/
[]	API/data model/error handling dokumentálva (ha releváns).	docs/03_design/*
[]	Van 10+ automata teszt, és futnak CI-ban.	docs/04_quality/* + CI
[]	Threat model + privacy/licensing leírás megvan.	docs/05_security_ops/*
[]	Deploy runbook + observability + 2 incident scenario megvan.	docs/05_security_ops/*
[]	AI manifest + prompt log + verification log megvan (5-10 verifikáció).	docs/07_ai/*
[]	Önértékelés kitöltve kategóriánként.	docs/06_release/ vagy külön fájl

6.1 Gyors ellenőrzés 15 perc alatt

Cél: 15 perc alatt derüljön ki, hogy a repo és a termék „kívülről” vállalható-e.

- Klónozd friss mappába a repót (vagy nyiss 'clean' workspace-t), és kövesd a README Quickstartot. Jegyezd fel, hol akadsz el.

- Futtasd a teszteket a README alapján. Ha bármi manuális varázslat kell, írd be a dokumentációba.
- Nézd meg a CI utolsó futását a main/default branch-en. Zöld-e? Linkeld be a scorecardba.
- Ellenőrizd a secret hygiene-t: nincs `.env`/kulcs/jelszó commitolva; a `git grep` ne találjon token. (Ha van scan tool, futtasd.)
- Menj végig a Demo Script-en: 5-7 percben stabilan végig tudod-e kattintani/lefuttatni a fő flow-kat és 1-2 hibahelyzetet is?
- Nyisd meg a docs indexet, és nézd meg: minden fejezet kész, és a diagramok/ADR-ek naprakészek?

6.2 Kapu checklist

A kapu feltételek a minimum termékminőségi alapok. Ha itt 'NEM' van, akkor először ezt javítsd, mert a pontozás plafonos lehet (4.1).

Kapu feltétel	Státusz (Igen/Nem)	Evidence / link	Megjegyzés
Futtathatóság: Quickstart alapján indul	☐ Igen / ☐ Nem		
CI zöld a main/default branch-en	☐ Igen / ☐ Nem		
Nincs secret a repo-ban (audit/scan)	☐ Igen / ☐ Nem		
Minimum tesztmix: 30+ automata teszt	☐ Igen / ☐ Nem		
AI átláthatóság: manifest+prompt+verification	☐ Igen / ☐ Nem		

6.3 100 pontos készütségi scorecard

Töltsd ki a táblákat. Minden sorban állítsd be az állapotot:

- 0.0 = nincs (Not started)
- 0.5 = részben kész (Partial, hiányzik evidence vagy a megoldás nem stabil)
- 1.0 = kész (Done, evidence linkelve)

Ha egy sor tényleg nem releváns, jelöld N/A-nak, és írd le röviden miért + mi a helyettesítő minőségi artefakt.

A) Product és scope - 12 pont

Pont	Követelmény	Állapot (0/0.5/1/N/A)	Evidence / link
2	Vision kész és konkrét: célfelhasználó/persona, értékajánlat, non-goals, top kockázatok (3.B / 01_product/vision.md).		

2	Scope Contract: MVP story-k + elfogadási kritériumok + scope fegyelem (3.B / 01_product/scope_contract.md).		
2	Capability Map kitöltve 6+ képességgel, státusszal és evidence linkekkel (3.C).		
2	UX flow-k dokumentáltak: 2-3 fő flow + error/empty state (3.D).		
2	Mérőszámok/metrics: mit mérnél és miért (01_product/metrics.md).		
2	Ismert korlátok + roadmap/tech debt: mi maradt kompromisszum és mi a következő lépés (bármely docs helyen).		

B) Képesség-szélesség - 10 pont

Pont	Követelmény	Állapot (0/0.5/1/N/A)	Evidence / link
2	Legalább 6 capability, ebből minimum 3 'termékesítő' (auth, CI, logging, error UX, deploy, monitoring, stb.).		
2	Minden Done capability-hez van konkrét evidence link (kép, endpoint, CI log, stb.).		
2	Minden Done capability-hez van kapcsolt teszt vagy teszt bizonyíték (unit/integration/e2e).		
2	Edge case-ek és hibák is capability szinten kezeltek (nem csak happy path).		
2	A capability map-ben a Planned/Partial elemek is őszintén jelöltek (nincs „késznek hazudás”).		

C) Architektúra és döntések - 13 pont

Pont	Követelmény	Állapot (0/0.5/1/N/A)	Evidence / link
3	C4 Context + Container diagram naprakészen (02_architecture/c4_context_container.md).		
3	Component/modul nézet: modulhatárok, felelősségek, adatáramlás		

	(02_architecture/c4_component.md vagy ekvivalens).		
3	Minimum 8 ADR: döntés, alternatívák, következmények, verifikáció (02_architecture/adr/*).		
2	Quality attributes + legalább 2 quality scenario (02_architecture/quality_attributes.md).		
2	Deployment view: hol fut mi, környezetek, komponensek kapcsolata (dokumentált).		

D) Engineering minőség - 15 pont

Pont	Követelmény	Állapot (0/0.5/1/N/A)	Evidence / link
3	Kódszerkezet és moduláris felépítés: rétegek, dependency irányok, ismétlés minimalizálva.		
3	Egységes hibakezelés + input validáció + konzisztens hibamodel (03_design/error_handling.md + kód).		
2	Konfiguráció és környezetek: beállítások különválasztása, config validáció, dokumentált eltérések.		
3	Teljesítmény baseline + 1 szűk keresztmetszet mérése és javítása (04_quality/performance.md vagy ekvivalens).		
2	Statikus minőségi kapuk: lint/format/typecheck (projektfüggő) és CI-ban fut.		
2	Platform-specifikus minőség: mobilnál erőforrás/UX guideline, webnél reszponzivitás/a11y minimum.		

E) Tesztelés és minőségi kapuk - 15 pont

Pont	Követelmény	Állapot (0/0.5/1/N/A)	Evidence / link
2	Teszt stratégia és teszt riport kitöltve, futtatás leírva (04_quality/test_strategy.md + test_report.md).		

4	30+ automata teszt értelmes mixben (unit+integration+e2e/contract), legalább 5 negatív teszttel.		
3	CI gating: a tesztek kötelezően futnak, és a main branch zöld; flake-ek kezelve vagy dokumentálva.		
2	Integrációs tesztek valós függőségekkel (DB/container, API stub) - nem csak mock minden.		
2	E2E/contract jellegű teszt a core flow-ra (UI teszt vagy API contract).		
2	Plusz minőségi teszt: performance smoke / security jellegű teszt / property-based / snapshot (projekthez igazítva).		

F) DevOps és üzemeltetés - 15 pont

Pont	Követelmény	Állapot (0/0.5/1/N/A)	Evidence / link
3	CI pipeline komplett: build + lint/format + teszt + artifact/image (projekthez igazítva).		
2	Reprodukálható build/dependency kezelés (lockfile, container build, build script).		
3	Deploy leírás + környezetek: hogyan lesz demo/prod jellegű futás, konfigurációval együtt.		
2	Rollback terv (leírva) és legalább egyszer végiggondolva/kipróbálva.		
3	Observability baseline: strukturált log + healthcheck + 3 metrika definíció (05_security_ops/observability.md).		
2	Runbook: 2 incident scenario + teendők, hibaokok diagnosztikája (05_security_ops/deploy_runbook.md).		

G) Security, privacy, licenc - 10 pont

Pont	Követelmény	Állapot (0/0.5/1/N/A)	Evidence / link

3	Threat model 6+ tétellel + konkrét mitigációk beépítve (05_security_ops/threat_model.md).		
2	Secret hygiene rendben: nincs titok, van `.env.example`, és van scan/audit evidence.		
2	AuthN/AuthZ modell dokumentált és tesztelt (jogosultság megtagadás tesztek is).		
2	Dependency vulnerability ellenőrzés (audit/scan) + kezelési terv (frissítés, pinning).		
1	Privacy + licensing: kezelt adatok, AI adatküldés szabály, licenc alapok (05_security_ops/privacy_licensing.md).		

H) AI engineering érettség - 10 pont

Pont	Követelmény	Állapot (0/0.5/1/N/A)	Evidence / link
2	AI manifest: eszközök és használati területek (07_ai/ai_manifest.md).		
2	Prompt log: 10-20 kulcsprompt kontextussal + kimenet linkkel (07_ai/prompt_log.md).		
3	Verification log: legalább 10 verifikáció, ebből minimum 3 tesztel és 2 méréssel/PoC-val (07_ai/verification_log.md).		
2	AI output integráció: AI által írt kód csak teszt + review után mehet main-re; checklist dokumentálva.		
1	Tanulságok: hol tévedett az AI, milyen guardrail-eket raktál be (dokumentált).		

Bónusz - 10 pont

Pont	Követelmény	Állapot (0/0.5/1/N/A)	Evidence / link
2	Feature flag / konfigurációs kapcsoló (release kontroll).		
2	Offline/queue/retry stratégia vagy resilience pattern (projekthez igazítva).		
2	Accessibility vagy i18n extra, ténylegesen		

	kipróbálható.		
2	Rate limiting / abuse prevention / anti-automation védelem (API-nál).		
2	Rövid post-mortem egy valós hibáról: ok, hatás, javítás, preventív teszt.		

6.4 Pontszám értelmezése

A scorecard célja nem az, hogy „szépen nézzen ki”, hanem hogy fókuszot adjon. A tipikus értelmezés (irányadó):

- 0-49 pont: prototípus. Alap dokumentáció és minőségi kapuk hiányosak.
- 50-69 pont: MVP jelleg. Fő funkciók megvannak, de termékesítés (teszt/ops/security) még gyenge.
- 70-84 pont: jó irány. A termék már közelít a release-hez, de van néhány magas kockázatú hiány.
- 85-94 pont: release candidate. A legtöbb termékminőségi terület rendben, a leadás stabil.
- 95-100 pont (+bónusz): kifejezetten érett, „production-minded” megoldás.

Ha elakadsz: mindig a legnagyobb kockázatú hiányt javítsd először (secret, teszt, deploy, observability, auth). Az új feature ritkán javít pontot, ha a termékminőség alapjai hiányoznak.

7. Mellékletek: sablonok

Másold ezeket a sablonokat a repo docs/ mappájába, és töltsd ki őket a saját projektedre szabva.

7.1 ADR sablon (docs/02_architecture/adr/000X-title.md)

Másold és töltsd ki minden nagy döntéshez. A cél a visszakereshetőség, nem a hossz.

```
# 000X - [Döntés címe]

Dátum: YYYY-MM-DD
Státusz: Proposed | Accepted | Deprecated

## Context
- Mi a probléma?
- Milyen kényszerek vannak (idő, tech, csapat, platform)?
- Milyen minőségi attribútumot érint (pl. security, performance)?

## Decision
- Egy mondatban: mit választottunk?

## Alternatives
1) Alternatíva A - előny/hátrány
2) Alternatíva B - előny/hátrány

## Consequences
- Pozitív következmények
- Negatív következmények / kockázatok
- Mit kell figyelni a megvalósítás során?
```

```
## Verification
- Hogyan ellenőrizzük? (teszt/mérés/PoC)
- Evidence link: [link]
```

7.2 Threat model sablon (STRIDE táblázat)

Minimum: 6 sor. Nem kell tökéletes, de legyen konkrét és a rendszeredre szabott.

Fenyegetés (STRIDE)	Leírás	Hatás	Valószínűség	Mitigáció	Verification (bizonyíték)

7.3 Verification log sablon

Minimum 5-10 bejegyzés. A cél: AI állítások ellenőrzése.

AI állítás/javaslat	Kockázat	Ellenőrzés módja	Eredmény (pass/fail + output)	Következtetés / változás

7.4 Teszt riport sablon (rövid)

```
# Test Report

## Környezet
- OS:
- Runtime/SDK:
- DB:
- Teszt adat:

## Teszt suite-ek és futtatás
- Unit: `...`
```

- Integration: `...`
- E2E/Contract: `...`

Legutolsó futás eredménye

- Dátum:
- Eredmény: PASS/FAIL
- CI link:

Ismert hiányosságok

- Mi nincs letesztelve és miért?

Flaky / instabil tesztek

- Van? Ha igen, hogyan kezeled?

7.5 Demo script sablon

A demo legyen rövid, stabil és a scope contract alapján épüljön fel (ne improvizáció).

```
# Demo Script (5-7 perc)
```

- 1) 30 mp - Mi a probléma és kinek szól? (Vision 1 mondat)
- 2) 2-3 perc - Fő user flow #1 (happy path)
- 3) 1 perc - Fő user flow #2 vagy admin/extra
- 4) 1 perc - Hibakezelés (pl. rossz input vagy hálózat)
- 5) 30-60 mp - Termékminőség: tesztek, CI, deploy/observability
- 6) 30 mp - Következő lépés / roadmap (non-goals és planned capability-k)

7.6 Metrics sablon

Nem kell éles analitika, de legyen világos, mit mérnél egy valódi terméknel.

Metrika típusa	Név	Definíció	Hogyan mérném?
North Star			
Guardrail			
Guardrail			
Guardrail			